Reševanje Maximum stable set problema z prilagojenim Petford Welsh algoritmom

Sergej Pelko
pelko.sergej@gmail.com

Novo mesto, September 2026

1 Uvod

V tem poročilu bom predstavil algoritem z reševanje Maximum stable set problema(MSSP) z uporabo prilagojenega Petford Welsh algoritma(PW).

2 Naloga in algoritem

Maximum stable set problem. Vzemimo graf $G = (V, E)$ z množico vozlišč $V = \{v_1, v_2, \ldots, v_n\}$, stabilna množica $S \subseteq V$ je podmnožica vozlišč, za katero velja, da za vsaki dve vozlišči $v_i, v_j \in S$ ne obstaja povezava $(v_i, v_j) \in E$. Iščemo takšno stabilno množico S, ki je največja po velikosti.
Množico S lahko predstavimo z vektorjem $x \in \{0,1\}^n$, za katerega velja $x_i = 1$, če in samo če je vozlišče $v_i \in S$. MSSP lahko formuliramo kot:

$$\alpha(G) = \max \left\{ e^T x \mid x \in \{0,1\}^n,\ x_i x_j = 0,\ \forall (v_i, v_j) \in E \right\}$$

Hamiltonijan za tak problem je:

$$H = -A \sum_{v_i \in V} x_i + B \sum_{(v_i, v_j) \in E} x_i x_j$$

Prvi člen maksimizira velikost stabilne množice, drugi člen pa kaznuje vse pare vozlišč, ki se nahajajo v stabilni množici in so povezani s povezavo.
Algoritem. Algoritem, ki ga bomo uporabili za reševanje MSSP je prilagojena verzija Petford Welsh algoritma(PW).

Require: Graf $G = (V, E)$; nagrado A, kazen B (kjer $B > A$); baza b; število iteracij max_iters

$x_v \leftarrow 0\ \forall v \in V$ ▷ *Začnemo z prazno množico*
$S^\star \leftarrow \emptyset$; best $\leftarrow 0$
for $t = 1 \rightarrow max_iters$ **do**
    Izberemo naključno vozlišče $u \in V$
    $X_{u,1} \leftarrow B \sum_{v \in N(u)} x_v - A$
    $X_{u,0} \leftarrow 0$
    Spremenimo stanje x_u z verjetnostjo $p_i \propto b^{-X_{u,i}}$
    if x je stabilna množica in $\sum_{v \in V} x_v >$ best **then**
        $S^\star \leftarrow \{v \in V : x_v = 1\}$; best $\leftarrow \sum_{v \in V} x_v$
return $S^\star$

3 Rezultati

3.1 Instance

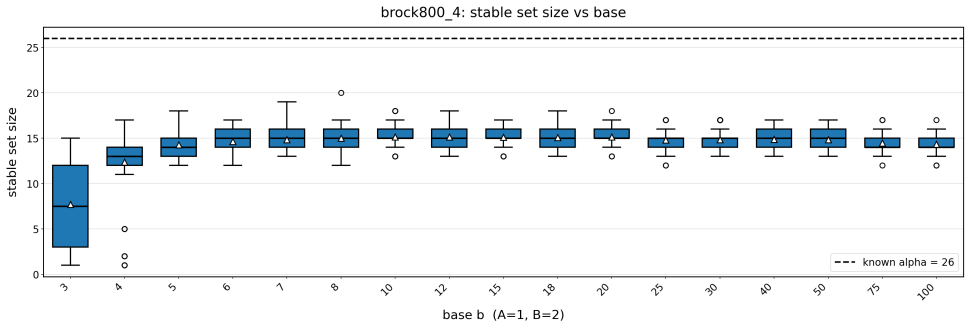
Za testiranje algoritma sem izbral nehaj različnih setov instanc, ki so lepo povzeti v članku [5]. Vsak komplet instanc ima svoje značilnosti, a vse so zasnovane tako, da je težko najti optimalno rešitev.

- **DIMACS** - 37 grafov iz izziva DIMACS [3], ki so uporabljeni v članku [4].
- **BHOSLIB** - 36 grafov s skritimi optimalnimi rešitvami [8].
- **EVIL** - 40 majhnh grafov z namerno skritimi optimalnimi rešitvami v naključnih grafih iz članka [7], naloženi iz [10].
- **Coding theory** - 32 grafov iz članka [6], za katere ne vemo optimalne rešitve.
- **PACE** - 95 grafov iz PACE 2019 izziva [1]. Vzeti so bili izzivi iz setov DIMACS, BHOSLIB,... Vsi imajo znano optimalno rešitev. [2]
- **Quantum** 61 instanc z največ 128 vozlišči, ki so zgrajene tako, da je energijska reža med osnovnim in prvim vzbujenim stanjem majhna, kar pomeni, da je težko najti optimalno rešitev s kvantnim računalnikom. [9]

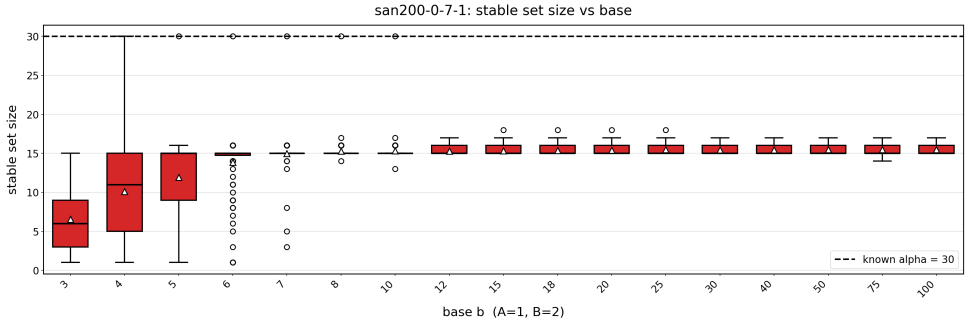
Najprej sem se osredotočil na DIMACS set in so rezultati do poglavja 4 prikazani le za te instance, kasneje pa sem algoritem preizkusil še na ostalih setih instanc.

3.2 Konstantni parametri

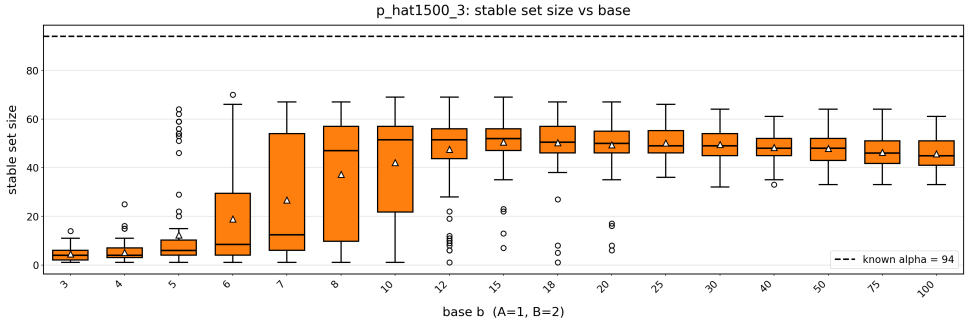
Učinkovitost algoritma je zelo odvisna od izbire parametrov A, B in b. Zaradi preceješnje različnosti instanc, ne moremo najti vrednosti parametrov, ki bi bile optimalne za vse primere, lahko pa se približno omejimo na nek interval vrednosti, kjer algoritem najde dobre rezultate. V nekaterih primerih so rešitve precej lahke za najti, in lahko uporabimo večjo bazo, kar vodi do manj naključnih odločitev in hitrejšega plezanja k optimalni rešitvi, medtem ko je v drugih primerih potrebno preiskati večji del prostora in je zato potrebna manjša baza.
Najprej poglejmo, kakšne rezultate dobimo, za fiksne vrednosti parametrov A, B in b. Vzel sem par testnih instanc za katere vem, da algoritem ponavadi ne najde optimalne rešitve in za vsako sem ponovil 100 ponovitev z različnimi vrednostmi parametrov B/A in b. Slika 1 prikazuje rezultate v odvisnosti od baze b pri fiksnih vrednostih $A = 1.0$ in $B = 2.0$, vidimo lahko, da ima baza zelo velik vpliv na rezultate. Za majhne baze je sistem preveč kaotičen in ne najde optimalne rešitve, za previsoke baze pa sistem preišče premajhen del prostora in se zatakne v lokalnem optimumu. To je lepo prikazano na primeru 1d, kjer algoritem najde optimalno rešitev le pri vmesnih vrednostih baze.
Razmerje B/A ima podoben vpliv, kot lahko vidimo na sliki 2, kjer sem fiksiral bazo $b = 8$ in spreminjal razmerje B/A. Tu pri vrednostih razmerja nad 4.0 skoraj ni razlik. Vzemimo, bazo $b = 8$ in razmerje $B/A = 4.0$. V tem primeru je verjetnost da prazno vozlišče sprejmemo v stabilno množico ≈ 0.889, medtem ko je verjetnost da vozlišče, ki je povezano z vsaj enim vozliščem v stabilni množici sprejmemo ≈ 0.001, medtem ko je verjetnos za vozlišče z dvema povezavama praktično 0. To pomeni, da algoritem sprejema le nova vozlišča in dobimo neke vrste naključni greedy algoritem.
Slika 3 prikazuje rezultate v odvisnosti od baze b in razmerja B/A, kjer je z barvo označena povprečna velikost stabilne množice, ki jo algoritem najde. Vidimo, da so najboljši rezultati doseženi v območju, kjer nobeden od parametrov ni premajhen ali prevelik. Če nevemo nič o instanci in moramo uporabiti eno vrednost parametrov je najbolj optimalna izbira $A = 1.0$, $B = 2.0$ in $b = 8$.



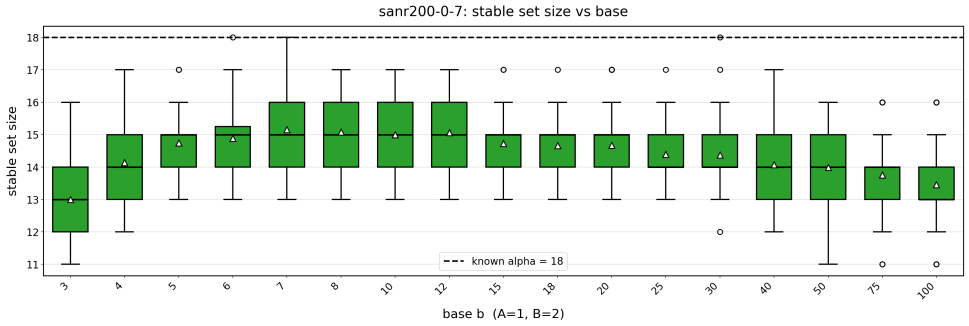
(a) brock800_4



(b) san200-0-7-1

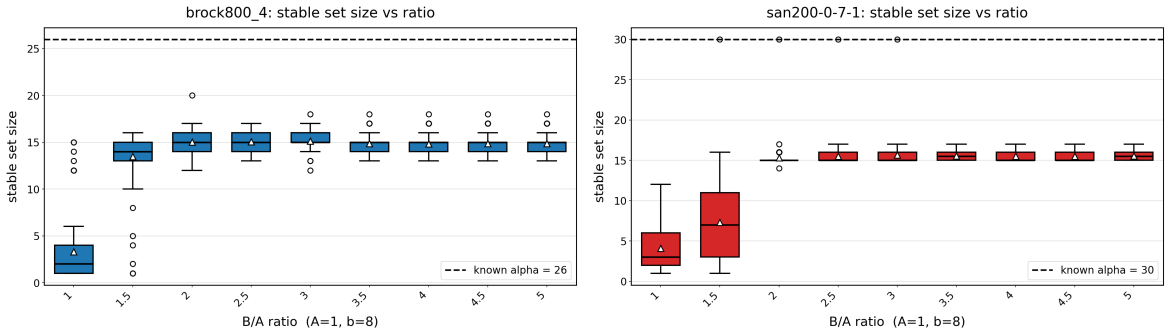


(c) p_hat1500_3



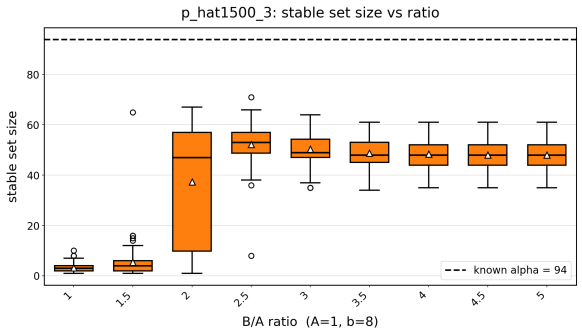
(d) sanr200-0-7

Slika 1: Rezultati v odvisnosti od baze b pri fiksnih vrednostih $A = 1.0$ in $B = 2.0$.

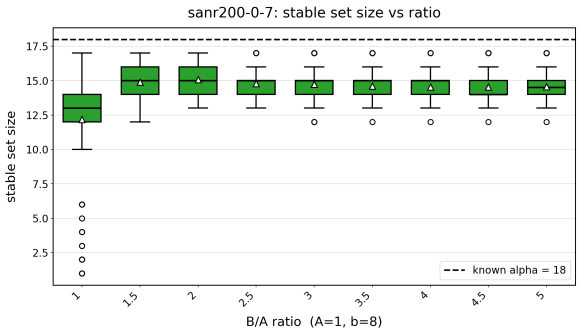


(a) brock800_4

(b) san200-0-7-1

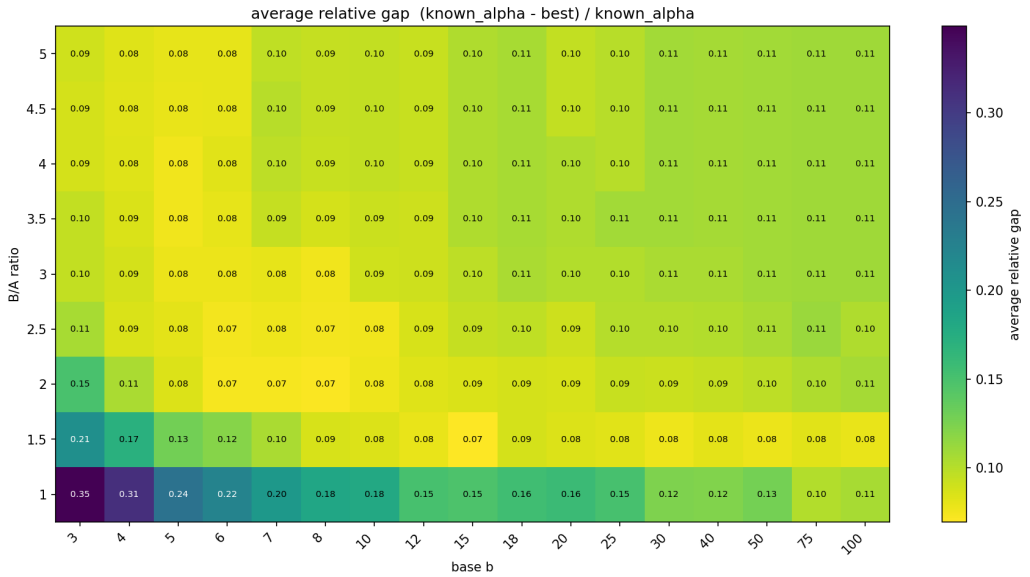


(c) p_hat1500_3



(d) sanr200-0-7-1

Slika 2: Rezultati v odvisnosti od razmerja B/A pri fiksni vrednosti baze $b = 4$.



Slika 3: Rezultati v odvisnosti od baze b in razmerja B/A.

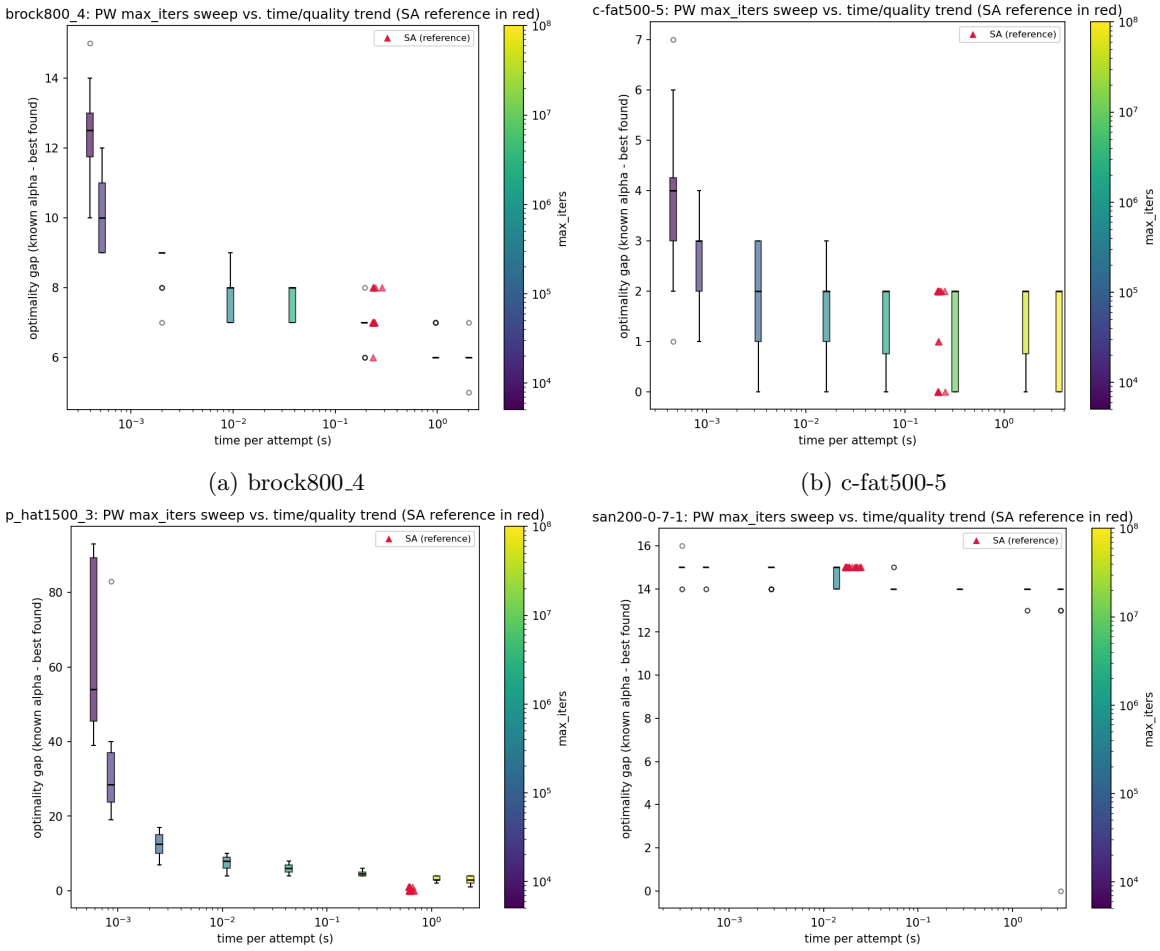
Nato sem s temi parametri izvedel algoritem na vseh instancah iz DIMACS seta, rezultati pa so prikazani v tabeli 1. Takšen algoritem, brez kakršnihkoli prilagoditev najde optimalno rešitev za 29 od 37 instanc. Stolpci si sledijo po naslednjem vrstnem redu: ime instance, število vozlišč, število povezav, število iteracij, število ponovitev, najboljša najdena stabilna množica, povprečna velikost stabilne množice, čas izvajanja v sekundah, najboljša znana stabilna množica, razlika med najboljšo najdeno in najboljšo znano stabilno množico.

instance	n	m	max_iters	num_attempts	best	mean	time_sec	known_alpha	gap
C125.9	125	787	1000000	100	34	34.00	2.76	34	0
MANN_a9	45	72	1000000	100	16	16.00	2.80	16	0
brock200-4	200	6811	1000000	100	17	15.95	1.94	17	0
brock800_1	800	112095	1000000	100	20	18.32	2.15	23	3
brock800_2	800	111434	1000000	100	20	18.48	1.97	24	4
brock800_3	800	112267	1000000	100	19	18.33	1.94	25	6
brock800_4	800	111957	1000000	100	20	18.45	1.94	26	6
c-fat200-1	200	18366	1000000	100	12	11.07	2.02	12	0
c-fat200-2	200	16665	1000000	100	24	22.25	2.35	24	0
c-fat200-5	200	11427	1000000	100	58	57.17	3.18	58	0
c-fat500-1	500	120291	1000000	100	14	12.59	1.97	14	0
c-fat500-2	500	115611	1000000	100	26	25.12	2.37	26	0
c-fat500-5	500	101559	1000000	100	64	62.54	3.31	64	0
dsjc125.5	125	3859	1000000	100	10	10.00	1.94	10	0
dsjc125.9	125	789	1000000	100	34	33.99	2.40	34	0
evil-N120-p98-chv12x10	120	545	1000000	100	20	20.00	2.52	20	0
evil-N120-p98-myc5x24	120	236	1000000	100	48	48.00	2.87	48	0
evil-N121-p98-myc11x11	121	508	1000000	100	22	22.00	2.47	22	0
evil-N125-p98-s3m25x5	125	873	1000000	100	20	20.00	2.24	20	0
hamming6_2	64	192	1000000	100	32	32.00	3.61	32	0
hamming6_4	64	1312	1000000	100	4	4.00	2.40	4	0
johnson16_2_4	120	1680	1000000	100	8	8.00	2.52	8	0
johnson8_2_4	28	168	1000000	100	4	4.00	2.77	4	0
johnson8_4_4	70	560	1000000	100	14	14.00	2.73	14	0
keller4	171	5100	1000000	100	11	11.00	2.54	11	0
p-hat500-1	500	93181	1000000	100	9	8.99	2.37	9	0
p_hat1500_1	1500	839327	1000000	100	11	10.16	2.55	12	1
p_hat1500_2	1500	555290	1000000	100	64	61.79	2.97	65	1
p_hat1500_3	1500	277006	1000000	100	91	87.62	2.83	94	3
paley101	101	2525	1000000	100	5	5.00	2.56	5	0
paley61	61	915	1000000	100	5	5.00	2.59	5	0
paley73	73	1314	1000000	100	5	5.00	2.49	5	0
paley89	89	1958	1000000	100	5	5.00	2.48	5	0
paley97	97	2328	1000000	100	6	6.00	2.43	6	0
san200-0-7-1	200	5970	1000000	100	30	15.77	3.48	30	0
san200-0-7-2	200	5970	1000000	100	14	12.81	3.57	18	4
sanr200-0-7	200	6032	1000000	100	18	17.95	2.50	18	0

Tabela 1: Rezultati izvajanja algoritma na testnih instancah. Za vse primere so bili uporabljeni parametri $A = 1.0$, $B = 2.0$, baza $b = 8$. Število ponovitev in število iteracij pa sta bili konstantni.

3.3 Čas izvajanja in optimalnost rezultatov

Število iteracij in ponovitev v prejšnji tabeli je bilo izbrano precej arbitrarno. Ker je to glavni faktor kako dobro rešitev bo algoritem našel bom v nadaljevanju preveril vpliv tega. Slika 4 prikazuje optimalnost rešitve v odvisnosti od časa izvajanja ene ponovitve algoritma, barva pa prikazuje število iteracij. Izbral sem nekaj različnih vrednosti parametra *max_iters* in za vsako izvedel 20 ponovitev algoritma, rezultati za vsako število iteracij so prikazani z boxplotom.



(a) brock800_4

(b) c-fat500-5

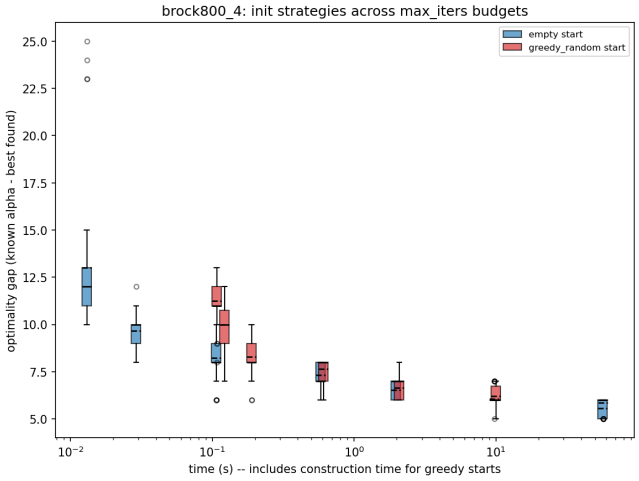
(c) p_hat1500_3

(d) san200-0-7-1

Slika 4: Optimalnost rešitve v odvisnosti od časa izvajanja ene ponovitve algoritma, barva pa prikazuje število iteracij.

3.4 Začetek na greedy rešitvi

Algoritem zmeraj začne z prazno stabilno množico in potem postopoma dodaja vozlišča. Hitrost bi lahko izboljšali, če začnemo z že obstoječo stabilno množico, ki je morda blizu optimalne rešitve. Zato sem dodal možnost, da algoritem najprej izvede greedy algoritem, ki najde neko stabilno množico in potem začne z algoritmom. Začnemo torej z prazno množico in najprej naključno dodajamo vozlišča, ki niso povezana s povezavo z že izbrano množico, dokler ne pridemo do neke stabilne množice, ki jo potem uporabimo kot začetno stanje algoritma. Primerjavo med začetkom z greedy rešitvijo in začetkom z prazno množico prikazuje spodnja slika 5, kjer je prikazana enaka stvar, kot na prejšnjih slikah, le da je sedaj z barvo označen način začetka algoritma. Modra barva označuje začetek z prazno množico, oranžna pa označuje začetek z greedy rešitvijo. Vidimo, da takšen postopek ne izboljša najdene rešitve, edina razlika je, da začetni korak traja dlje časa, zato so rešitve za majhen čas izvajanja slabše, kot če bi začeli z prazno množico. Tu je največji faktor to, da je koda za inicializacijo slabše optimizirana.



Slika 5: Optimalnost rešitve v odvisnosti od časa izvajanja ene ponovitve algoritma, barva pa prikazuje način začetka algoritma.

3.5 Primerjava najboljšega stanja z končnim

Algoritem si tekom poteka shranjuje najboljšo rešitev, ki jo je našel. Opcija, ki jo lahko dodamo, je da namesto najboljše rešitve, vzamemo končno stanje, ki ga algoritem najde. Ta množica ponavadi sploh ni stabilna, ampak je lahko blizu optimalne rešitve, ki jo lahko popravimo tako, da postopoma odstranjujemo vozlišča, ki so povezana s povezavo. Primerjavo teh dveh postopkov prikazuje spodnja tabela 2, kjer lahko vidimo, da je končno stanje zmeraj slabše od najboljše najdene stabilne množice, zato bom še naprej uporabljal prvotno verzijo algoritma.

instance	n	m	max_iters	num_attempts	known_alpha	avg_best	avg_repaired	max_best	max_repaired	gap_best	gap_repaired
C125.9	125	787	62500	50	34	25.68	16.98	28	23	6	11
MANN_a9	45	72	22500	50	16	15.20	9.70	16	14	0	2
brock200-4	200	6811	100000	50	17	12.60	7.60	14	11	3	6
brock800_1	800	112095	400000	50	23	15.36	10.46	17	13	6	10
brock800_2	800	111434	400000	50	24	15.34	10.24	17	13	7	11
brock800_3	800	112267	400000	50	25	15.20	9.84	17	13	8	12
brock800_4	800	111957	400000	50	26	15.34	10.00	17	13	9	13
c-fat200-1	200	18366	100000	50	12	11.28	5.06	12	9	0	3
c-fat200-2	200	16665	100000	50	24	18.94	9.80	21	16	3	8
c-fat200-5	200	11427	100000	50	58	40.94	25.92	44	37	14	21
c-fat500-1	500	120291	250000	50	14	12.28	5.88	14	9	0	5
c-fat500-2	500	115611	250000	50	26	21.18	12.10	24	17	2	9
c-fat500-5	500	101559	250000	50	64	44.56	29.50	48	43	16	21
dsjc125.5	125	3859	62500	50	10	8.50	4.76	9	7	1	3
dsjc125.9	125	789	62500	50	34	25.70	16.36	28	24	6	10
evil-N120-p98-chv12x10	120	545	60000	50	20	20.00	15.60	20	18	0	2
evil-N120-p98-myc5x24	120	236	60000	50	48	37.74	26.42	41	34	7	14
evil-N121-p98-myc11x11	121	508	60500	50	22	22.00	15.54	22	20	0	2
evil-N125-p98-s3m25x5	125	873	62500	50	20	19.48	13.24	20	17	0	3
hamming6_2	64	192	32000	50	32	24.58	13.46	27	19	5	13
hamming6_4	64	1312	32000	50	4	4.00	2.74	4	4	0	0
johnson16_2_4	120	1680	60000	50	8	8.00	5.90	8	7	0	1
johnson8_2_4	28	168	14000	50	4	4.00	2.74	4	4	0	0
johnson8_4_4	70	560	35000	50	14	11.94	6.24	14	11	0	3
keller4	171	5100	85500	50	11	9.74	5.96	11	9	0	2
p-hat500-1	500	93181	250000	50	9	7.98	4.34	9	7	0	2
p_hat1500_1	1500	839327	750000	50	12	9.02	5.58	10	7	2	5
p_hat1500_2	1500	555290	750000	50	65	42.34	29.88	47	36	18	29
p_hat1500_3	1500	277006	750000	50	94	57.70	41.76	62	52	32	42
paley101	101	2525	50500	50	5	5.00	4.04	5	5	0	0
paley61	61	915	30500	50	5	5.00	3.52	5	5	0	0
paley73	73	1314	36500	50	5	5.00	3.68	5	5	0	0
paley89	89	1958	44500	50	5	5.00	4.20	5	5	0	0
paley97	97	2328	48500	50	6	6.00	4.24	6	6	0	0
san200-0-7-1	200	5970	100000	50	30	15.46	12.88	16	15	14	15
san200-0-7-2	200	5970	100000	50	18	12.88	10.80	13	12	5	6
sanr200-0-7	200	6032	100000	50	18	14.06	8.54	16	11	2	7

Tabela 2: Primerjava med najboljšo najdeno stabilno množico in končnim stanjem algoritma. Za vse primere so bili uporabljeni parametri $A = 1.0$, $B = 2.0$, baza $b = 8$. Končnica *_best* označuje najboljšo najdeno stabilno množico, končnica *_repaired* pa označuje stabilno množico, ki je bila popravljena iz končnega stanja algoritma.

3.6 Ugibanje optimalne rešitve

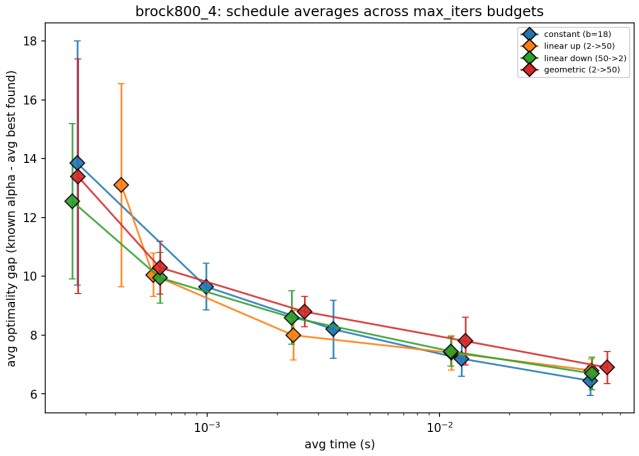
Za poljubno podmnožico lahko izračunamo približno optimalno rešitev, ki jo lahko algoritem najde. Če tej podmnožici odštejemo vozlišča, ki so povezana s povezavo, dobimo stabilno množico, ki jo lahko uporabimo za približek optimalne rešitve (ugib = velikost stanja − število povezav v stanju). Algoritmu torej dodamo parameter, ki sledi stanju, ki da najboljšo oceno. Če je ta ocena boljša od najboljše najdene stabilne množice, potem temu stanju postopoma odstranjujemo vozlišča, dokler ne dobimo stabilne množice, ki jo lahko uporabimo kot najboljšo rešitev. Razlika med tema postopkoma je prikazana v spodnji tabeli 3, kjer lahko vidimo, da ocena nikoli ne izboljša najboljše najdene stabilne množice, zato bom ostal pri prvotni verziji algoritma. Tu je prostor za izboljšavo, če bi začeli z naključno rešitvijo, kjer je možnost za najdenje boljše ocene večja.

instance	n	m	max_iters	num_attempts	best	mean	best_estimated	time_sec	known_alpha	gap	estimated_gap
C125.9	125	787	62500	500	34	33.70	34	1.40	34	0	0
MANN_a9	45	72	22500	500	16	16.00	16	0.10	16	0	0
brock200-4	200	6811	100000	500	17	14.91	17	1.01	17	0	0
brock800_1	800	112095	400000	100	20	17.92	20	1.08	23	3	3
brock800_2	800	111434	400000	100	20	18.09	20	0.91	24	4	4
brock800_3	800	112267	400000	100	20	18.01	20	1.34	25	5	5
brock800_4	800	111957	400000	100	19	18.07	19	0.96	26	7	7
c-fat200-1	200	18366	100000	500	12	11.00	12	0.73	12	0	0
c-fat200-2	200	16665	100000	500	24	22.26	24	1.17	24	0	0
c-fat200-5	200	11427	100000	500	58	57.14	58	1.28	58	0	0
c-fat500-1	500	120291	250000	200	14	12.54	14	1.08	14	0	0
c-fat500-2	500	115611	250000	200	26	25.08	26	0.95	26	0	0
c-fat500-5	500	101559	250000	200	64	62.43	64	1.74	64	0	0
dsjc125.5	125	3859	62500	500	10	9.45	10	0.58	10	0	0
dsjc125.9	125	789	62500	500	34	33.08	34	0.73	34	0	0
evil-N120-p98-chv12x10	120	545	60000	500	20	20.00	20	0.23	20	0	0
evil-N120-p98-myc5x24	120	236	60000	500	48	47.30	48	0.88	48	0	0
evil-N121-p98-myc11x11	121	508	60500	500	22	22.00	22	0.23	22	0	0
evil-N125-p98-s3m25x5	125	873	62500	500	20	20.00	20	0.23	20	0	0
hamming6_2	64	192	32000	500	32	32.00	32	0.14	32	0	0
hamming6_4	64	1312	32000	500	4	4.00	4	0.12	4	0	0
johnson16_2_4	120	1680	60000	500	8	8.00	8	0.22	8	0	0
johnson8_2_4	28	168	14000	500	4	4.00	4	0.06	4	0	0
johnson8_4_4	70	560	35000	500	14	14.00	14	0.14	14	0	0
keller4	171	5100	85500	500	11	10.99	11	0.48	11	0	0
p-hat500-1	500	93181	250000	200	9	8.59	9	0.82	9	0	0
p_hat1500_1	1500	839327	750000	100	11	10.08	11	1.69	12	1	1
p_hat1500_2	1500	555290	750000	100	64	61.73	64	2.09	65	1	1
p_hat1500_3	1500	277006	750000	100	91	88.09	91	1.74	94	3	3
paley101	101	2525	50500	500	5	5.00	5	0.18	5	0	0
paley61	61	915	30500	500	5	5.00	5	0.11	5	0	0
paley73	73	1314	36500	500	5	5.00	5	0.13	5	0	0
paley89	89	1958	44500	500	5	5.00	5	0.16	5	0	0
paley97	97	2328	48500	500	6	6.00	6	0.17	6	0	0
san200-0-7-1	200	5970	100000	500	30	15.35	30	1.40	30	0	0
san200-0-7-2	200	5970	100000	500	14	12.45	14	1.36	18	4	4
sanr200-0-7	200	6032	100000	500	18	17.02	18	0.97	18	0	0

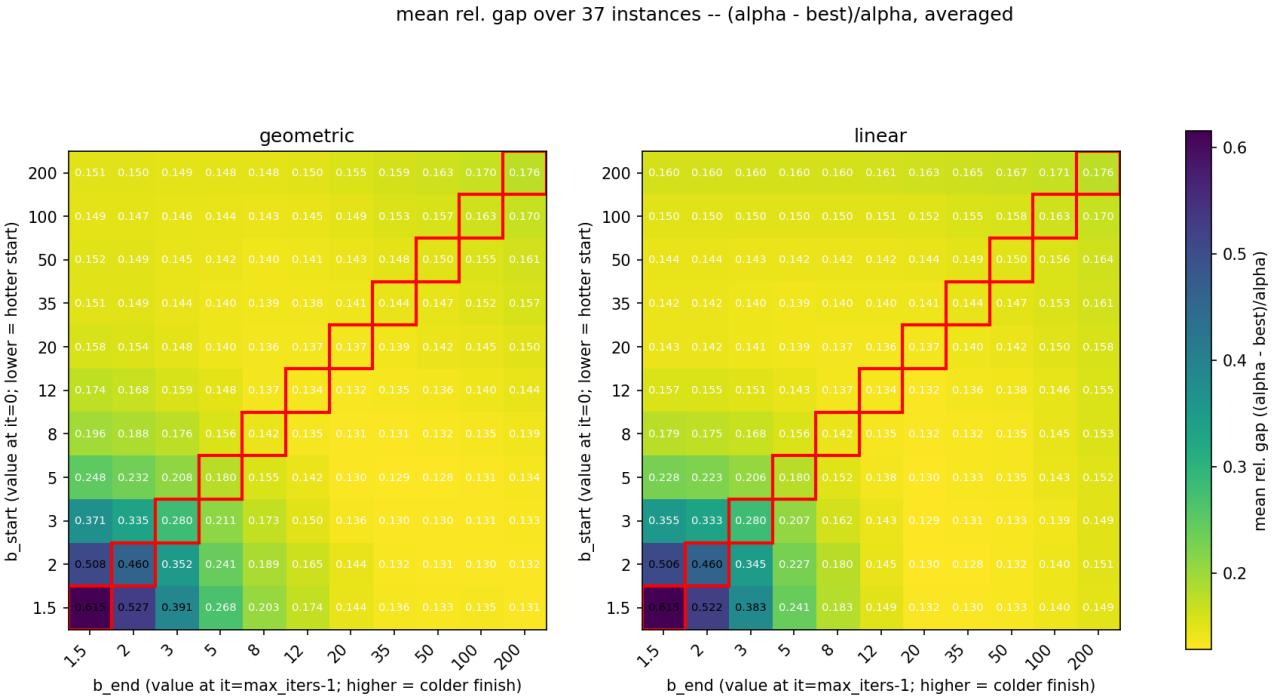
Tabela 3: Primerjava med najboljšo najdeno stabilno množico in najboljšo oceno, ki jo je algoritem našel. Za vse primere so bili uporabljeni parametri $A = 1.0$, $B = 2.0$, baza $b = 8$.

3.7 Spreminjanje baze

Zaradi precejšnje različnosti instanc, ponavadi ni mogoče najti optimalne baze in razmerja B/A za vse primere. V nekaterih primerih je fazni prostor precej manj kaotičen in lahko rešitev zelo hitro najdemo z visoko bazo, medtem ko je drugje potrebno preiskati večji del prostora in je zato potrebna manjša baza. Rezultate lahko izboljšamo, tako da med izvajanjem algoritma spreminjamo bazo. Tu sem si izbral dve različni strategiji, ena je preprosto linearno spreminjanje baze od začetne do končne vrednosti, drugo pa je geometrijsko spreminjanje. V obeh primerih lahko strategijo popolnoma parametriziramo z začetno in končno vrednostjo baze. Spodnja slika 6 kaže povprečno odstopanje od optimalne rešitve v odvisnosti od časa izvajanja algoritma na eni izmed instanc, kjer je z barvo označena strategija spreminjanja baze, v legendi pa sta navedeni začetna in končna vrednost baze. Opazimo, da so v vseh primerih rešitve zelo podobne in da je odstopanje v merah napake.
Da niso vsi rezultati osnovani na eni specifični instanci, sem ponovil meritve še na vseh preostalih instancah in relativno odstopanje od optimalne rešitve povprečil. To sem ponovil za cel razpon začetnih in končnih vrednosti baze. Spodnja slika 7 prikazuje točno to, kjer barva prikazuje optimalnost rešitve, ki jo algoritem najde. Opazimo, da geometrijsko spreminjanje baze da malenkost boljše rezultate, prav tako pa se boljše obnesejo urniki z toplejšimi štartom in hladnejšim koncem, kot obratno. Razlike med strategijami pa so skoraj zanemarljive in lahko dobimo zelo dobre rezultate kar z fiksno bazo.



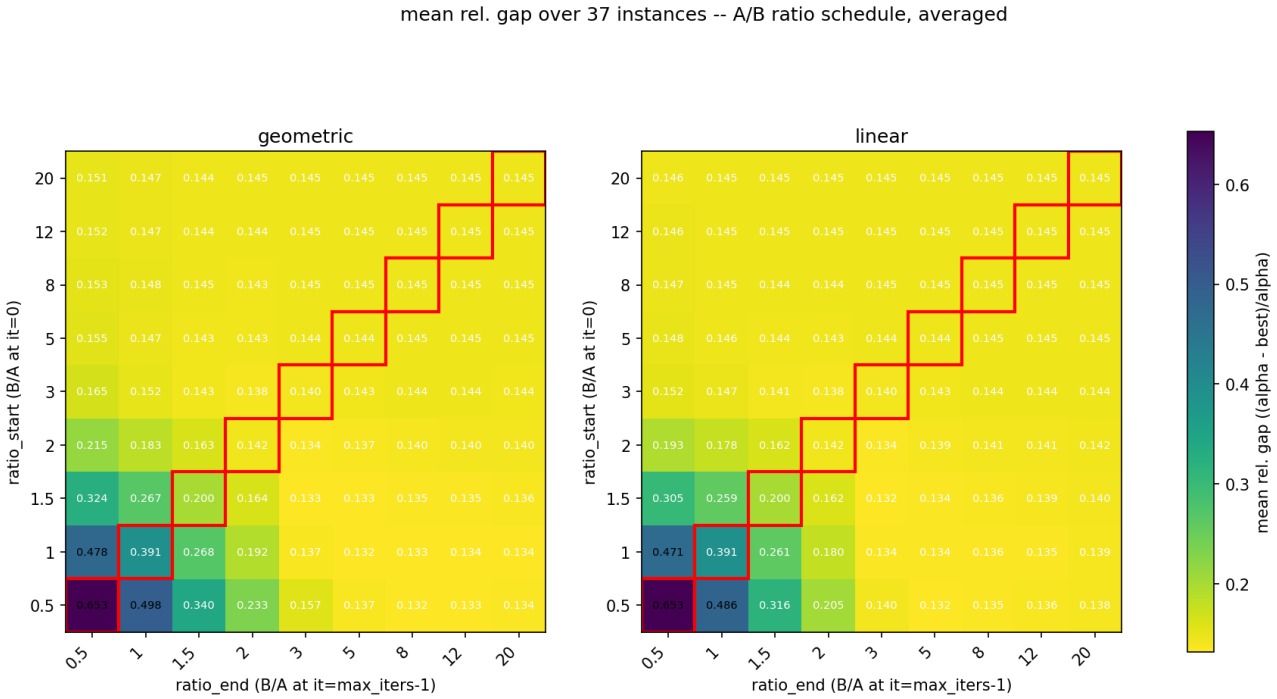
Slika 6: Povprečno odstopanje od optimalne rešitve za različne strategije spreminjanja baze.



Slika 7: Relativno odstopanje od optimalne rešitve v odvisnosti od začetne in končne vrednosti baze. Rdeči kvadrati označujejo fiksno bazo.

3.8 Spreminjanje razmerja B/A

Podobno stvar lahko ponovimo tudi za razmerje B/A. Ideja je, da bi začeli z večjim razmerjem B/A, da bi na začetku, ko imamo precej prazno množico, hitreje dodajali vozlišča, potem pa bi razmerje B/A znižali, da bi prioritizirali stabilnost množice. Spodnja slika 8 kaže relativno povprečno odstopanje od optimalne rešitve za različne strategije spreminjanja razmerja B/A. Razlike med tem in fiksnim razmerjem skoraj ni, najboljše vrednosti pa so dosežene pri razmerju $1.5 - 2$.



Slika 8: Relativno odstopanje od optimalne rešitve v odvisnosti od začetne in končne vrednosti razmerja B/A. Rdeči kvadrati označujejo fiksno razmerje B/A.

4 Solver

Očitno spreminjanje baze in razmerja B/A tekom izvajanja ene ponovitve algoritma ne prinese boljših rezultatov, kot če bi uporabili fiksne vrednosti. Hkrati pa ne obstajajo optimalne vrednosti parametrov, ki bi bile dobre za vse primere. Zato sem se odločil, da namesto tega, da za vse ponovitve algoritma uporabim iste parametre, bom za vsako ponovitev izbral druge vrednosti. Za bazo sem uzel vrednosti $b \in \{1.5, 2.0, 3.0, 5.0, 8.0, 10.0, 15.0, 20.0, 30.0, 50.0\}$, za razmerje B/A pa vrednosti $r \in \{1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0, 4.5, 5.0, 5.5\}$. Nato naredim kartezičnni produkt teh dveh množic in za vsako ponovitev po vrsti izberem en par parametrov. Število iteracij sem propriconalno povečal glede na število vozlišč v grafu. Najprej sem za vsako instanco izbral število iteracij $max_iters = 10 \cdot |V|$, in po končanem izvajanju preveril rešitev. Če je bila rešitev optimalna, sem končal, sicer pa sem povečal število iteracij za faktor 10 in ponovil postopek, dokler nisem našel optimalne rešitve ali pa presegel čas izvajanja 10 minut. Za število ponovitev sem izbral 100. Tabele spodaj prikazuje rezultate izvajanja algoritma na vseh testnih instancah.

Tabela 4: Rezultati izvajanja algoritma na instancah iz množice DIMACS z naraščajočim proračunom iteracij. Za vsak primer so prikazani najboljša najdena velikost neodvisne množice (best), znana vrednost (known alpha), najmanjše število iteracij na poskus, pri katerem je bila najboljša velikost dosežena (num_iters), in skupni čas računanja po vseh stopnjah proračuna (time_sec). Uporabljeni so bili $A = 1.0$, $B \in [1, 5.5]$ in baze $b \in [1.5, 50]$.

instance	n	m	best	known alpha	num_iters	time_sec	gap	rel_gap
brock200-4	200	6811	17	17	20000000	46.28	0	0.0000
brock800_1	800	112095	21	23	8000000	182.48	2	0.0870
brock800_2	800	111434	21	24	80000000	182.38	3	0.1250
brock800_3	800	112267	22	25	80000000	182.21	3	0.1200
brock800_4	800	111957	21	26	8000000	182.61	5	0.1923
c-fat200-1	200	18366	12	12	2000	0.01	0	0.0000
c-fat200-2	200	16665	24	24	2000	0.01	0	0.0000
c-fat200-5	200	11427	58	58	2000	0.01	0	0.0000

Nadaljevanje na naslednji strani

instance	n	m	best	known alpha	num_iters	time_sec	gap	rel_gap
c-fat500-1	500	120291	14	14	5000	0.02	0	0.0000
c-fat500-2	500	115611	26	26	5000	0.03	0	0.0000
c-fat500-5	500	101559	64	64	5000	0.03	0	0.0000
C125.9	125	787	34	34	12500	0.04	0	0.0000
dsjc125.5	125	3859	10	10	12500	0.03	0	0.0000
dsjc125.9	125	789	34	34	12500	0.04	0	0.0000
evil-N120-p98-chv12x10	120	545	20	20	1200	0.01	0	0.0000
evil-N120-p98-myc5x24	120	236	48	48	12000	0.04	0	0.0000
evil-N121-p98-myc11x11	121	508	22	22	1210	0.01	0	0.0000
evil-N125-p98-s3m25x5	125	873	20	20	1250	0.00	0	0.0000
hamming6_2	64	192	32	32	640	0.00	0	0.0000
hamming6_4	64	1312	4	4	640	0.00	0	0.0000
johnson16_2_4	120	1680	8	8	1200	0.00	0	0.0000
johnson8_2_4	28	168	4	4	280	0.00	0	0.0000
johnson8_4_4	70	560	14	14	700	0.00	0	0.0000
keller4	171	5100	11	11	1710	0.01	0	0.0000
MANN_a9	45	72	16	16	450	0.00	0	0.0000
p-hat500-1	500	93181	9	9	50000	0.11	0	0.0000
p_hat1500_1	1500	839327	12	12	15000000	34.91	0	0.0000
p_hat1500_2	1500	555290	65	65	1500000	4.28	0	0.0000
p_hat1500_3	1500	277006	94	94	15000000	38.41	0	0.0000
paley101	101	2525	5		1010	228.98		
paley61	61	915	5		610	141.66		
paley73	73	1314	5		730	168.21		
paley89	89	1958	5		890	202.29		
paley97	97	2328	6		970	218.87		
san200-0-7-1	200	5970	30	30	20000	0.06	0	0.0000
san200-0-7-2	200	5970	18	18	200000	0.54	0	0.0000
sanr200-0-7	200	6032	18	18	20000	0.05	0	0.0000

Tabela 5: Rezultati izvajanja algoritma na instancah iz množice BHOSLIB z naraščajočim proračunom iteracij. Za vsak primer so prikazani najboljša najdena velikost neodvisne množice (best), znana vrednost (known alpha), najmanjše število iteracij na poskus, pri katerem je bila najboljša velikost dosežena (num_iters), in skupni čas računanja po vseh stopnjah proračuna (time_sec). Uporabljeni so bili $A = 1.0$, $B \in [1, 5.5]$ in baze $b \in [1.5, 50]$.

instance	n	m	best	known alpha	num_iters	time_sec	gap	rel_gap
frb100-40	4000	572774	88		40000000	93.40		
frb30-15-1	450	17827	30	30	450000	1.02	0	0.0000
frb30-15-2	450	17874	30	30	4500000	10.22	0	0.0000
frb30-15-3	450	17809	29	30	4500000	104.45	1	0.0333
frb30-15-4	450	17831	30	30	4500000	10.21	0	0.0000
frb30-15-5	450	17794	30	30	45000000	104.18	0	0.0000
frb35-17-1	595	27856	33	35	5950000	136.80	2	0.0571
frb35-17-2	595	27847	35	35	59500000	137.19	0	0.0000
frb35-17-3	595	27931	35	35	59500000	137.68	0	0.0000
frb35-17-4	595	27842	34	35	59500000	137.55	1	0.0286
frb35-17-5	595	28143	35	35	59500000	137.39	0	0.0000
frb40-19-1	760	41314	39	40	7600000	177.00	1	0.0250
frb40-19-2	760	41263	39	40	76000000	177.13	1	0.0250
frb40-19-3	760	41095	38	40	7600000	175.26	2	0.0500
frb40-19-4	760	41605	39	40	76000000	174.90	1	0.0250

Nadaljevanje na naslednji strani

instance	n	m	best	known alpha	num_iters	time_sec	gap	rel_gap
frb40-19-5	760	41619	39	40	76000000	174.54	1	0.0250
frb45-21-1	945	59186	43	45	94500000	219.96	2	0.0444
frb45-21-2	945	58624	43	45	94500000	217.43	2	0.0444
frb45-21-3	945	58245	43	45	94500000	217.93	2	0.0444
frb45-21-4	945	58549	43	45	94500000	217.87	2	0.0444
frb45-21-5	945	58579	43	45	94500000	217.13	2	0.0444
frb50-23-1	1150	80072	48	50	115000000	269.38	2	0.0400
frb50-23-2	1150	80851	48	50	115000000	266.95	2	0.0400
frb50-23-3	1150	81068	48	50	115000000	266.93	2	0.0400
frb50-23-4	1150	80258	48	50	1150000	267.29	2	0.0400
frb50-23-5	1150	80035	48	50	115000000	271.73	2	0.0400
frb53-24-1	1272	94227	50	53	127200000	302.57	3	0.0566
frb53-24-2	1272	94289	50	53	127200000	301.21	3	0.0566
frb53-24-3	1272	94127	50	53	127200000	302.36	3	0.0566
frb53-24-4	1272	94308	50	53	127200000	301.25	3	0.0566
frb53-24-5	1272	94226	50	53	127200000	300.24	3	0.0566
frb59-26-1	1534	126555	56	59	15340000	353.64	3	0.0508
frb59-26-2	1534	126163	56	59	153400000	357.48	3	0.0508
frb59-26-3	1534	126082	57	59	153400000	361.65	2	0.0339
frb59-26-4	1534	127011	56	59	15340000	364.10	3	0.0508
frb59-26-5	1534	125982	55	59	153400000	360.48	4	0.0678

Tabela 6: Rezultati izvajanja algoritma na instancah iz množice PACE 2019 z naraščajočim proračunom iteracij. Za vsak primer so prikazani najboljša najdena velikost neodvisne množice (best), znana vrednost (known alpha), najmanjše število iteracij na poskus, pri katerem je bila najboljša velikost dosežena (num_iters), in skupni čas računanja po vseh stopnjah proračuna (time_sec). Uporabljeni so bili $A = 1.0$, $B \in [1, 5.5]$ in baze $b \in [1.5, 50]$.

instance	n	m	best	known alpha	num_iters	time_sec	gap	rel_gap
vc-exact_001	6160	40207	3533	3574	61600000	236.66	41	0.0115
vc-exact_003	60541	48418	47638	48351	60541000	233.42	713	0.0147
vc-exact_005	200	798	71	71	20000	0.07	0	0.0000
vc-exact_007	8794	10121	4245	4397	87940000	325.34	152	0.0346
vc-exact_009	38452	174645	16790	17104	38452000	148.94	314	0.0184
vc-exact_011	9877	25973	4841	4896	98770000	378.92	55	0.0112
vc-exact_013	45307	36697	36169	36697	45307000	170.42	528	0.0144
vc-exact_015	53610	42940	42309	42940	53610000	205.22	631	0.0147
vc-exact_017	23541	34233	11091	11459	23541000	91.36	368	0.0321
vc-exact_019	200	862	70	70	200000	0.62	0	0.0000
vc-exact_021	24765	19655	19394	19655	2476500	89.09	261	0.0133
vc-exact_023	27717	133665	11495	11704	27717000	104.00	209	0.0179
vc-exact_025	23194	18295	18052	18295	23194000	83.27	243	0.0133
vc-exact_027	65866	52435	51678	52435	65866000	258.13	757	0.0144
vc-exact_029	13431	16234	6621	6809	134310000	517.82	188	0.0276
vc-exact_031	200	813	64	64	200000	0.59	0	0.0000
vc-exact_033	4410	6885	1641	1685	4410000	156.89	44	0.0261
vc-exact_035	200	864	67	67	200000	0.61	0	0.0000
vc-exact_037	198	808	67	67	198000	0.60	0	0.0000
vc-exact_039	6795	10620	2521	2595	67950000	244.09	74	0.0285
vc-exact_041	200	1023	61	61	2000000	5.84	0	0.0000
vc-exact_043	200	841	61	61	200000	0.59	0	0.0000
vc-exact_045	200	1020	63	63	200000	0.59	0	0.0000

Nadaljevanje na naslednji strani

instance	n	m	best	known alpha	num_iters	time_sec	gap	rel_gap
vc-exact_047	200	1093	60	60	200000	0.58	0	0.0000
vc-exact_049	200	933	64	64	200000	0.60	0	0.0000
vc-exact_051	200	1098	60	60	200000	0.58	0	0.0000
vc-exact_053	200	1026	61	61	200000	0.59	0	0.0000
vc-exact_055	200	938	66	66	200000	0.60	0	0.0000
vc-exact_057	200	1160	58	58	200000	0.57	0	0.0000
vc-exact_059	200	961	63	63	200000	0.59	0	0.0000
vc-exact_061	200	931	65	65	2000000	5.95	0	0.0000
vc-exact_063	200	1011	62	62	200000	0.58	0	0.0000
vc-exact_065	200	1011	62	62	200000	0.59	0	0.0000
vc-exact_067	200	1174	57	57	20000	0.06	0	0.0000
vc-exact_069	200	1083	60	60	200000	0.58	0	0.0000
vc-exact_071	200	952	64	64	200000	0.60	0	0.0000
vc-exact_073	200	1078	61	61	200000	0.59	0	0.0000
vc-exact_075	26300	41500	9826	10000	26300000	99.87	174	0.0174
vc-exact_079	26300	41500	9826	10000	26300000	99.95	174	0.0174
vc-exact_081	199	1091	58	58	199000	0.58	0	0.0000
vc-exact_083	200	1172	56	56	200000	0.56	0	0.0000
vc-exact_085	11470	17408	4319	4436	114700000	421.18	117	0.0264
vc-exact_087	13590	21240	5033	5190	135900000	513.55	157	0.0303
vc-exact_089	57316	77978	22133	23178	57316000	227.62	1045	0.0451
vc-exact_091	200	1163	55	55	200000	0.57	0	0.0000
vc-exact_093	200	1162	57	57	200000	0.57	0	0.0000
vc-exact_095	15783	24663	5851	6028	15783000	57.82	177	0.0294
vc-exact_097	18096	28281	6702	6911	18096000	67.54	209	0.0302
vc-exact_099	26300	41500	9826	10000	26300000	100.08	174	0.0174
vc-exact_101	26300	41500	9826	10000	26300000	100.09	174	0.0174
vc-exact_103	15783	24663	5841	6028	15783000	57.84	187	0.0310
vc-exact_105	26300	41500	9826	10000	26300000	100.01	174	0.0174
vc-exact_107	13590	21240	5035	5190	135900000	502.88	155	0.0299
vc-exact_109	66992	90970	25871	27110	66992000	269.26	1239	0.0457
vc-exact_113	26300	41500	9826	10000	26300000	99.90	174	0.0174
vc-exact_115	18096	28281	6695	6911	18096000	67.50	216	0.0313
vc-exact_117	18096	28281	6692	6911	18096000	67.51	219	0.0317
vc-exact_119	18096	28281	6689	6911	18096000	67.47	222	0.0321
vc-exact_121	18096	28281	6697	6911	18096000	67.46	214	0.0310
vc-exact_123	26300	41500	9826	10000	26300000	100.01	174	0.0174
vc-exact_125	26300	41500	9826	10000	26300000	100.01	174	0.0174
vc-exact_127	18096	28281	6697	6911	18096000	67.68	214	0.0310
vc-exact_129	15783	24663	5842	6028	15783000	57.77	186	0.0309
vc-exact_131	2980	5360	1016	1060	29800000	100.67	44	0.0415
vc-exact_133	15783	24663	5838	6028	15783000	57.74	190	0.0315
vc-exact_135	26300	41500	9826	10000	26300000	100.04	174	0.0174
vc-exact_137	26300	41500	9825	10000	26300000	100.01	175	0.0175
vc-exact_139	18096	28281	6697	6911	18096000	67.52	214	0.0310
vc-exact_141	18096	28281	6694	6911	18096000	67.42	217	0.0314
vc-exact_143	18096	28281	6691	6911	18096000	67.54	220	0.0318
vc-exact_145	18096	28281	6698	6911	18096000	67.62	213	0.0308
vc-exact_147	18096	28281	6693	6911	18096000	67.55	218	0.0315
vc-exact_149	26300	41500	9826	10000	26300000	99.94	174	0.0174
vc-exact_151	15783	24663	5841	6028	15783000	57.85	187	0.0310
vc-exact_153	29076	45570	10865	11083	29076000	111.06	218	0.0197
vc-exact_155	26300	41500	9826	10000	26300000	100.01	174	0.0174

Nadaljevanje na naslednji strani

instance	n	m	best	known alpha	num_iters	time_sec	gap	rel_gap
vc-exact_157	2980	5360	1014	1060	29800000	100.64	46	0.0434
vc-exact_159	18096	28281	6695	6911	18096000	67.58	216	0.0313
vc-exact_161	138141	227241	48799	51496	13814100	86.56	2697	0.0524
vc-exact_163	18096	28281	6691	6911	18096000	67.80	220	0.0318
vc-exact_165	18096	28281	6691	6911	18096000	67.63	220	0.0318
vc-exact_167	15783	24663	5848	6028	15783000	57.80	180	0.0299
vc-exact_169	4768	8576	1622	1696	47680000	162.74	74	0.0436
vc-exact_171	18096	28281	6697	6911	18096000	67.53	214	0.0310
vc-exact_173	56860	77264	21953	23004	56860000	226.73	1051	0.0457
vc-exact_175	3523	6446	1182	1241	35230000	119.51	59	0.0475
vc-exact_177	5066	9112	1723	1802	50660000	173.45	79	0.0438
vc-exact_179	15783	24663	5839	6028	15783000	57.85	189	0.0314
vc-exact_181	18096	28281	6691	6911	18096000	67.51	220	0.0318
vc-exact_183	72420	118362	25544	27082	72420000	281.45	1538	0.0568
vc-exact_185	3523	6446	1180	1241	35230000	119.44	61	0.0492
vc-exact_187	4227	7734	1413	1489	42270000	148.17	76	0.0510
vc-exact_189	7400	13600	2469	2600	74000000	257.33	131	0.0504
vc-exact_191	4579	8378	1530	1613	45790000	156.83	83	0.0515
vc-exact_193	7030	12920	2341	2470	70300000	244.81	129	0.0522

Tabela 7: Rezultati izvajanja algoritma na instancah iz množice Quantum z naraščajočim proračunom iteracij. Za vsak primer so prikazani najboljša najdena velikost neodvisne množice (best), znana vrednost (known alpha), najmanjše število iteracij na poskus, pri katerem je bila najboljša velikost dosežena (num_iters), in skupni čas računanja po vseh stopnjah proračuna (time_sec). Uporabljeni so bili $A = 1.0$, $B \in [1, 5.5]$ in baze $b \in [1.5, 50]$.

instance	n	m	best	known alpha	num_iters	time_sec	gap	rel_gap
a265032_1dc.64-c	64	543	10	10	6400	0.02	0	0.0000
a265032_1et.128-c	128	672	28	28	1280	0.01	0	0.0000
a265032_1et.64-c	64	264	18	18	640	0.00	0	0.0000
a265032_1tc.128-c	128	512	38	38	128000	0.44	0	0.0000
a265032_1tc.64-c	64	192	20	20	640	0.00	0	0.0000
a265032_1zc.128-c	128	1120	18	18	1280	0.00	0	0.0000
a265032_2dc.128-c	128	5173	5	5	1280	0.00	0	0.0000
evil-N100-p98-g136x10	100	56	80	80	1000	0.01	0	0.0000
evil-N100-p98-g150x10	100	27	90	90	1000	0.01	0	0.0000
evil-N100-p98-g50634x5	100	107	50	50	1000	0.01	0	0.0000
evil-N100-p98-myc5x20	100	189	40	40	10000	0.04	0	0.0000
evil-N100-p98-s3m25x4	100	672	16	16	1000	0.00	0	0.0000
evil-N108-p98-chv12x9	108	492	18	18	1080	0.00	0	0.0000
evil-N108-p98-g33896x6	108	524	42	42	10800	0.04	0	0.0000
evil-N108-p98-g33918x6	108	459	42	42	10800	0.04	0	0.0000
evil-N110-p98-g32559x10	110	106	80	80	11000	0.04	0	0.0000
evil-N110-p98-g32565x10	110	191	70	70	11000	0.04	0	0.0000
evil-N114-p98-g33920x6	114	562	48	48	11400	0.04	0	0.0000
evil-N115-p98-myc23x5	115	1015	10	10	1150	0.00	0	0.0000
evil-N120-p98-g136x12	120	83	96	96	1200	0.01	0	0.0000
evil-N120-p98-g50634x6	120	149	60	60	1200	0.01	0	0.0000
evil-N126-p98-g33896x7	126	612	49	49	12600	0.04	0	0.0000
evil-N126-p98-g33918x7	126	553	49	49	12600	0.04	0	0.0000
evil-N50-p98-s3m25x2	50	315	8	8	500	0.00	0	0.0000
evil-N55-p98-myc11x5	55	197	10	10	550	0.00	0	0.0000

Nadaljevanje na naslednji strani

instance	n	m	best	known alpha	num_iters	time_sec	gap	rel_gap
evil-N60-p98-chv12x5	60	245	10	10	600	0.00	0	0.0000
evil-N60-p98-g136x6	60	28	48	48	600	0.00	0	0.0000
evil-N60-p98-g150x6	60	11	54	54	600	0.00	0	0.0000
evil-N60-p98-g50634x3	60	50	30	30	600	0.00	0	0.0000
evil-N60-p98-myc5x12	60	90	24	24	600	0.00	0	0.0000
evil-N66-p98-g32559x6	66	50	48	48	660	0.00	0	0.0000
evil-N66-p98-g32565x6	66	86	42	42	6600	0.03	0	0.0000
evil-N69-p98-myc23x3	69	587	6	6	690	0.00	0	0.0000
evil-N72-p98-chv12x6	72	306	12	12	720	0.00	0	0.0000
evil-N72-p98-g33896x4	72	319	28	28	7200	0.03	0	0.0000
evil-N72-p98-g33918x4	72	283	28	28	7200	0.02	0	0.0000
evil-N75-p98-s3m25x3	75	483	12	12	750	0.00	0	0.0000
evil-N76-p98-g33920x4	76	349	32	32	760	0.00	0	0.0000
evil-N77-p98-myc11x7	77	300	14	14	770	0.00	0	0.0000
evil-N80-p98-g136x8	80	44	64	64	800	0.00	0	0.0000
evil-N80-p98-g150x8	80	21	72	72	800	0.00	0	0.0000
evil-N80-p98-g50634x4	80	81	40	40	800	0.00	0	0.0000
evil-N80-p98-myc5x16	80	124	32	32	800	0.00	0	0.0000
evil-N84-p98-chv12x7	84	360	14	14	840	0.00	0	0.0000
evil-N88-p98-g32559x8	88	95	64	64	8800	0.03	0	0.0000
evil-N88-p98-g32565x8	88	133	56	56	880	0.00	0	0.0000
evil-N90-p98-g33896x5	90	420	35	35	900	0.00	0	0.0000
evil-N90-p98-g33918x5	90	368	35	35	9000	0.04	0	0.0000
evil-N92-p98-myc23x4	92	782	8	8	920	0.00	0	0.0000
evil-N95-p98-g33920x5	95	434	40	40	9500	0.03	0	0.0000
evil-N96-p98-chv12x8	96	408	16	16	960	0.00	0	0.0000
evil-N99-p98-myc11x9	99	406	18	18	990	0.00	0	0.0000
hamming6-2	64	192	32		640	217.28		
hamming6-4	64	1312	4		640	146.01		
johnson16-2-4	120	1680	8		1200	275.92		
johnson8-4-4	70	560	14		700	176.24		

Tabela 8: Rezultati izvajanja algoritma na instancah iz množice EVIL z naraščajočim proračunom iteracij. Za vsak primer so prikazani najboljša najdena velikost neodvisne množice (best), znana vrednost (known alpha), najmanjše število iteracij na poskus, pri katerem je bila najboljša velikost dosežena (num_iters), in skupni čas računanja po vseh stopnjah proračuna (time_sec). Uporabljeni so bili $A = 1.0$, $B \in [1, 5.5]$ in baze $b \in [1.5, 50]$.

instance	n	m	best	known alpha	num_iters	time_sec	gap	rel_gap
chv12x10	120	545	20	20	1200	0.00	0	0.0000
chv12x15	180	944	30	30	1800	0.01	0	0.0000
chv12x20	240	1352	40	40	2400	0.01	0	0.0000
chv12x30	360	2378	60	60	3600	0.01	0	0.0000
chv12x40	480	3845	80	80	48000	0.13	0	0.0000
chv12x60	720	7415	120	120	7200000	18.26	0	0.0000
myc11x11	121	508	22	22	1210	0.01	0	0.0000
myc11x14	154	701	28	28	1540	0.01	0	0.0000
myc11x17	187	901	34	34	1870	0.01	0	0.0000
myc11x20	220	1130	40	40	2200	0.01	0	0.0000
myc11x23	253	1456	46	46	2530	0.01	0	0.0000
myc11x30	330	2035	60	60	33000	0.09	0	0.0000
myc11x40	440	3201	80	80	44000	0.12	0	0.0000

Nadaljevanje na naslednji strani

instance	n	m	best	known alpha	num_iters	time_sec	gap	rel_gap
myc11x50	550	4592	100	100	550000	1.45	0	0.0000
myc11x60	660	6302	120	120	6600000	17.03	0	0.0000
myc23x10	230	2263	20	20	2300	0.01	0	0.0000
myc23x15	345	3867	30	30	3450	0.01	0	0.0000
myc23x20	460	5664	40	40	4600	0.01	0	0.0000
myc23x25	575	7639	50	50	5750	0.01	0	0.0000
myc23x30	690	10078	60	60	6900	0.02	0	0.0000
myc23x6	138	1242	12	12	1380	0.01	0	0.0000
myc23x8	184	1764	16	16	1840	0.01	0	0.0000
myc5x100	500	2509	200	200	500000	1.60	0	0.0000
myc5x120	600	3663	240	240	600000	1.90	0	0.0000
myc5x140	700	4785	280	280	700000	2.22	0	0.0000
myc5x24	120	236	48	48	12000	0.04	0	0.0000
myc5x30	150	338	60	60	15000	0.05	0	0.0000
myc5x36	180	439	72	72	18000	0.06	0	0.0000
myc5x42	210	541	84	84	21000	0.07	0	0.0000
myc5x48	240	718	96	96	240000	0.79	0	0.0000
myc5x60	300	1033	120	120	300000	0.96	0	0.0000
myc5x80	400	1662	160	160	40000	0.19	0	0.0000
s3m25x10	250	2050	40	40	2500	0.01	0	0.0000
s3m25x15	375	3613	60	60	37500	0.10	0	0.0000
s3m25x20	500	5329	80	80	500000	1.25	0	0.0000
s3m25x25	625	7398	100	100	6250000	15.59	0	0.0000
s3m25x30	750	9813	120	120	7500000	18.50	0	0.0000
s3m25x5	125	873	20	20	1250	0.01	0	0.0000
s3m25x6	150	1102	24	24	1500	0.01	0	0.0000
s3m25x8	200	1550	32	32	2000	0.01	0	0.0000

Tabela 9: Rezultati izvajanja algoritma na instancah iz množice Coding theory z naraščajočim proračunom iteracij. Za vsak primer so prikazani najboljša najdena velikost neodvisne množice (best), znana vrednost (known alpha), najmanjše število iteracij na poskus, pri katerem je bila najboljša velikost dosežena (num_iters), in skupni čas računanja po vseh stopnjah proračuna (time_sec). Uporabljeni so bili $A = 1.0$, $B \in [1, 5.5]$ in baze $b \in [1.5, 50]$.

instance	n	m	best	known alpha	num_iters	time_sec	gap	rel_gap
1dc.1024	1024	24063	94	94	10240000	23.63	0	0.0000
1dc.128	128	1471	16	16	12800	0.03	0	0.0000
1dc.2048	2048	58367	172	174	204800000	559.41	2	0.0115
1dc.256	256	3839	30	30	256000	0.61	0	0.0000
1dc.512	512	9727	52	52	51200000	122.80	0	0.0000
1dc.64	64	543	10	10	6400	0.02	0	0.0000
1et.1024	1024	9600	168	171	102400000	269.85	3	0.0175
1et.128	128	672	28	28	1280	0.01	0	0.0000
1et.2048	2048	22528	300	316	20480000	54.06	16	0.0506
1et.256	256	1664	50	50	25600	0.07	0	0.0000
1et.512	512	4032	99	100	512000	135.75	1	0.0100
1et.64	64	264	18	18	640	0.00	0	0.0000
1tc.1024	1024	7936	192	196	102400000	279.96	4	0.0204
1tc.128	128	512	38	38	128000	0.37	0	0.0000
1tc.16	16	22	8	8	160	0.00	0	0.0000
1tc.2048	2048	18944	334	352	20480000	53.19	18	0.0511

Nadaljevanje na naslednji strani

instance	n	m	best	known alpha	num_iters	time_sec	gap	rel_gap
1tc.256	256	1312	63	63	25600000	72.45	0	0.0000
1tc.32	32	68	12	12	320	0.00	0	0.0000
1tc.512	512	3264	110	110	5120000	13.74	0	0.0000
1tc.64	64	192	20	20	640	0.00	0	0.0000
1tc.8	8	6	4	4	80	0.00	0	0.0000
1zc.1024	1024	16640	110	117	102400000	247.42	7	0.0598
1zc.128	128	1120	18	18	1280	0.00	0	0.0000
1zc.2048	2048	39424	188	210	204800000	553.20	22	0.1048
1zc.256	256	2816	36	36	25600	0.06	0	0.0000
1zc.4096	4096	92160	326	410	40960000	95.99	84	0.2049
1zc.512	512	6912	62	62	512000	1.20	0	0.0000
2dc.1024	1024	169162	16	16	102400000	237.31	0	0.0000
2dc.128	128	5173	5	5	1280	0.00	0	0.0000
2dc.2048	2048	504451	23	24	204800000	556.24	1	0.0417
2dc.256	256	17183	7	7	2560	0.01	0	0.0000
2dc.512	512	54895	11	11	512000	1.14	0	0.0000

Tabela spodaj prikazuje povzetke rezultatov za vsak set instanc posebej. Vidimo da sta seta quantum in EVIL precej lahka in algoritem najde optimalno rešitev za vse instance. Največ problema pa ima s seti PACE in bhoslib, kjer je rešitev povprečno slabša za $2-4\%$ od optimalne. Grafi iz seta PACE so precej veliki zato je pri tako majhnem času izvajanja pričakovano, da algoritem ne najde optimalne rešitve.

set	instances	optimal	$\leq 1\%$	$\leq 2\%$	$\leq 5\%$	$\leq 10\%$	$\leq 25\%$	avg time_sec	mean rel_gap
dimacs	32	28	28	28	28	29	32	49.04	0.0164
bhoslib	35	7	7	7	25	35	35	211.85	0.0351
PACE	95	26	26	49	89	95	95	107.13	0.0215
quantum	52	52	52	52	52	52	52	14.58	0.0000
EVIL	40	40	40	40	40	40	40	2.01	0.0000
coding_theory	32	22	23	25	27	30	32	102.45	0.0179
all	286	175	176	201	261	281	286	80.39	0.0153

5 Zaključek

Algoritem je precej hiter in da dobre rezultate. Zaenkrat večina predlaganih izboljšav ni prineslo boljših rezultatov, ampak bi se dalo še marsikaj poskusiti. V prihodnje bi lahko izboljšali algoritem za začetno stanje, da bi morda izboljšali rezultate, prav tako bi lahko dodali še kakšen postprocesing. Lahko bi tudi poskusil dodati boljšo oceno optimalne rešitve, kot le velikost trenutne množice minus število povezav v njej. Zaenkrat sem ločeno preizkušal spreminjanje baze in razmerja B/A, v prihodnje pa bi lahko poskusil tudi kombinirati oba postopka in videti, če bi to prineslo boljše rezultate. Razlika med tem in originalnim PW algoritmom je, da originalni ne izbere naključnega vozlišča, ampak izbere naključno slabo vozlišče. Morda bi lahko začeli z polno množico in na ta način odstranjevali vozlišča, dokler ne pridemo do stabilne rešitve. Prav tako bi lahko namesto naključnega izbora vozlišča, izbrali vozlišča po vrsti in tako zagotovili, da algoritem preišče cel graf enakomerno.

Literatura

[1] M. Ayaz Dzulfikar, Johannes K. Fichte, and Markus Hecher. The PACE 2019 parameterized algorithms and computational experiments challenge: The fourth iteration. In Bart M. P. Jansen and Jan Arne Telle, editors, *14th International Symposium on Parameterized and Exact Computation (IPEC 2019)*, volume 148 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 25:1–25:23, Dagstuhl, Germany, 2019. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[2] Johannes K. Fichte and Markus Hecher. PACE 2019 vertex cover instances and reference solutions. `https://github.com/daajoe/pace2019_vc_instances`, 2019. Accessed 23 September 2026.

[3] David S. Johnson and Michael A. Trick, editors. *Cliques, Coloring, and Satisfiability: Second DIMACS Implementation Challenge, October 11–13, 1993*, volume 26 of *DIMACS Series in Discrete Mathematics and Theoretical Computer Science*. American Mathematical Society, Providence, RI, 1996.

[4] Aljaž Krpan, Janez Povh, and Dunja Pucher. Quantum computing and the stable set problem. *Optimization Methods and Software*, 40(4):837–870, May 2025.

[5] Raffaele Marino, Lorenzo Buffoni, and Bogdan Zavalnij. A short review on the maximum clique problem algorithms with classical, ai, and quantum methods. *Communications Physics*, 9(1), 2026.

[6] N. J. A. Sloane. Challenge problems: Independent sets in graphs. `https://oeis.org/A265032/a265032.html`, 2000. Accessed 23 September 2026.

[7] Sándor Szabó and Bogdán Zaválnij. Benchmark problems for exhaustive exact maximum clique search algorithms. *Informatica*, 43(2):177–186, 2019.

[8] Ke Xu. BHOSLIB: Benchmarks with hidden optimum solutions for graph problems. `https://iridia.ulb.ac.be/~fmascia/maximum_clique/BHOSLIB-benchmark`. Accessed 23 September 2026.

[9] Bogdán Zaválnij. Benchmark graphs for quantum maximum clique algorithms. `https://github.com/zbogdan/qbenchmarks`. Accessed 23 September 2026.

[10] Bogdán Zaválnij. EVIL benchmark graphs for maximum clique search. `https://github.com/zbogdan/evil2`. Accessed 23 September 2026.